

MIVE: Markerless Inertial Vector Extraction for Monocular Quadrotor Pose Estimation

Julius Schneider

ETH Zurich, Switzerland

sjulius@ethz.ch

Abstract

Deep learning approaches for monocular quadrotor pose estimation and thrust vector regression are fundamentally bottlenecked by the difficulty of acquiring scalable, high-fidelity ground truth data in the physical world. Traditional solutions rely on either synthetic rendering, which suffers from a persistent sim-to-real domain gap, or physical fiducial markers (e.g., ArUco), which artificially bias visual feature representations. In this paper, we introduce Markerless Inertial Vector Extraction (MIVE), a novel data generation paradigm that leverages mobile Visual-Inertial Odometry (VIO) sensor fusion and steady-state hover kinematics to extract continuous 3D thrust vectors in real-world environments. To evaluate the efficacy of our pipeline against traditional methods, we utilize a single-stage deep learning baseline by extending YOLO11n to regress 3D thrust vectors. Our comparative study demonstrates that MIVE significantly mitigates the sim-to-real domain shift and prevents the shortcut learning induced by fiducial markers, providing actionable insights into scalable data generation for agile aerial tracking.

1 Introduction

For vision-based drone tracking, extracting a continuous 3D thrust vector from a single 2D RGB image remains a challenging problem. While architectural advancements in monocular 6DoF pose estimation have accelerated, ranging from multi-stage pipelines relying on geometric solvers [Sun et al., 2022, Rad and Lepetit, 2017, Shugurov et al., 2022, Wang et al., 2021, Labbé et al., 2022] to single-stage neural detectors [Bukschat and Vetter, 2020, Liu et al., 2023, Jung et al., 2025] and hybrid template-matching models like GigaPose [Nguyen et al., 2024], the primary limiting factor for learning-based pose estimation in aerial robotics remains the quality, scale, and realism of ground-truth annotations [Lachance et al., 2023].

Traditionally, researchers have bypassed the need for expensive physical motion-capture infrastructure by relying on synthetic datasets or physical fiducial markers. However, these methods introduce critical failure modes: synthetic data suffers from a persistent sim-to-real domain gap regarding optical characteristics and material rendering, while physical markers like ArUco boards risk biasing the neural network toward artificial, high-contrast geometric patterns, Scribano et al. [2026] leading to shortcut learning that fails to generalize to unmarked airframes.

To address these limitations, we propose Markerless Inertial Vector Extraction (MIVE), a data annotation framework leveraging mobile Visual-Inertial Odometry (VIO). By exploiting the aerodynamic force equilibrium of a quadrotor in a steady-state hover, we derive highly accurate, continuous 3D thrust vector annotations from physical multi-view imagery without external infrastructure.

The primary contribution of this work is the empirical benchmarking of three distinct data generation methodologies: synthetic simulation, fiducial marker tracking, and our novel MIVE pipeline. To facilitate this comparison, we extend a lightweight YOLO11n model to serve as a uniform perception benchmark, directly predicting spatial orientation in a single forward pass.

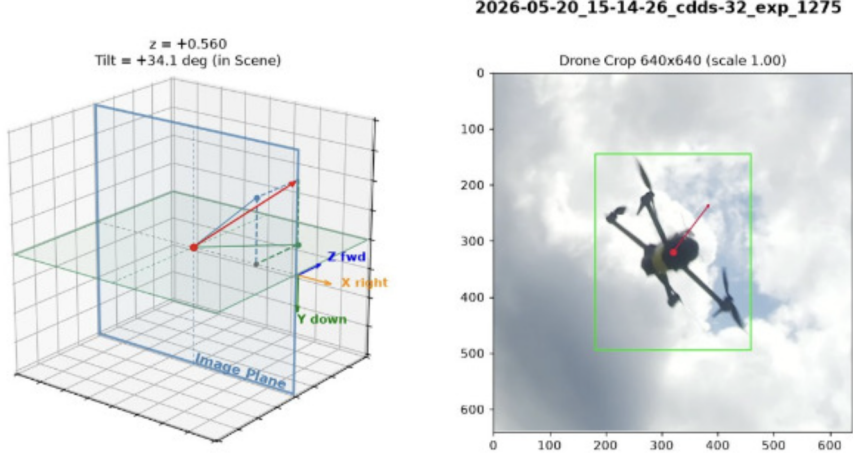


Figure 1: Qualitative validation on a physical quadrotor. The predicted 3D thrust vector (red) closely aligns with the ground-truth hover vector (blue) bounded by the estimated 2D bounding box.

2 Methodology

High-fidelity ground-truth data is crucial for robust pose estimation. We first introduce two well known data generation methodologies: synthetic simulation and fiducial marker tracking. We then introduce our novel MIVE approach. Subsequently, we detail our modified YOLO11 architecture and custom orientation loss formulation to evaluate the approaches.

2.1 Synthetic Data Generation via Simulation

To generate scalable ground-truth annotations without physical motion-capture infrastructure, we build a photorealistic synthetic data engine utilizing NVIDIA Isaac Sim and Omniverse Replicator. The pipeline renders 640×640 RGB images paired with synchronized 2D bounding boxes and exact continuous 3D thrust vectors.

2.1.1 Coordinate Transformations and Vector Derivation

The quadrotor’s primary thrust axis in its local body frame is defined as a unit vector aligned with its vertical symmetry axis, expressed in homogeneous coordinates as $\tilde{\mathbf{v}}_{\text{local}} = [0, 1, 0, 0]^\top$. To compute the ground-truth thrust vector in the camera reference frame, we perform a sequence of rigid-body coordinate transformations:

1. **Body-to-World Transformation:** The local thrust vector is mapped into world space using the drone’s 4×4 local-to-world transformation matrix $\mathbf{M}_{\text{world}} \in \mathbb{R}^{4 \times 4}$, extracted directly from the Universal Scene Description (USD) stage hierarchy:

$$\tilde{\mathbf{v}}_{\text{world}} = \mathbf{M}_{\text{world}} \tilde{\mathbf{v}}_{\text{local}}, \quad \hat{\mathbf{v}}_{\text{world}} = \frac{\mathbf{v}_{\text{world}}}{\|\mathbf{v}_{\text{world}}\|_2} \quad (1)$$

where $\mathbf{v}_{\text{world}} \in \mathbb{R}^3$ denotes the spatial components of $\tilde{\mathbf{v}}_{\text{world}}$.

2. **World-to-Camera Transformation:** The normalized world-frame thrust vector is transformed into the camera sensor frame using the camera view matrix $\mathbf{T}_{\text{cam}} \in \mathbb{R}^{4 \times 4}$:

$$\tilde{\mathbf{v}}_{\text{cam}} = \mathbf{T}_{\text{cam}} \begin{bmatrix} \hat{\mathbf{v}}_{\text{world}} \\ 0 \end{bmatrix}, \quad \hat{\mathbf{v}}_{\text{cam}} = \frac{\mathbf{v}_{\text{cam}}}{\|\mathbf{v}_{\text{cam}}\|_2} \quad (2)$$

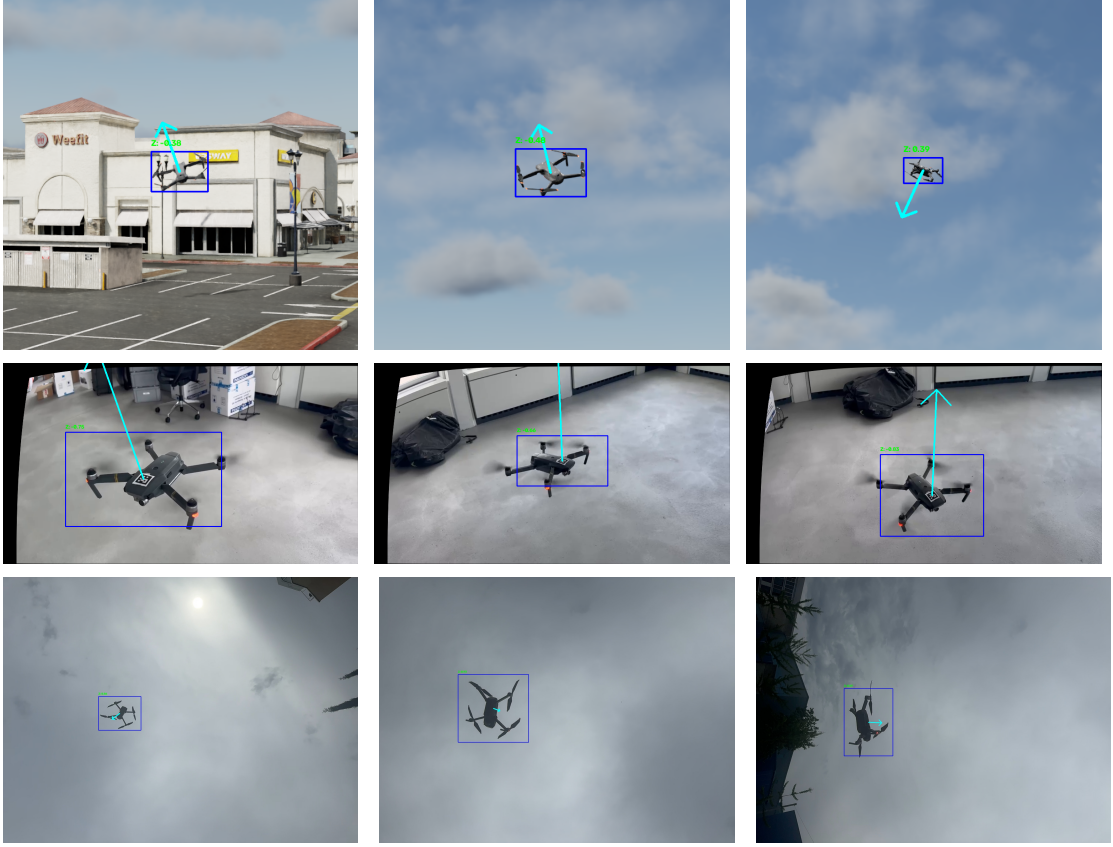


Figure 2: **Qualitative comparison of dataset generation methodologies.** *Top Row:* Synthetic renders from Isaac Sim, which exhibit a noticeable optical sim-to-real domain gap. *Middle Row:* Physical data utilizing ArUco markers, which risk artificially biasing the feature extractor toward high-contrast planar geometry. *Bottom Row:* Our proposed MIVE dataset, capturing the natural visual complexity, shading, and asymmetry of unmarked operational airframes in real-world environments.

3. OpenCV Convention Adjustment: Because graphics engines like USD/OpenGL use a right-handed coordinate system (X -right, Y -up, Z -backward), we apply a conversion matrix $\mathbf{A}_{CV} = \text{diag}(1, -1, -1, 1)$ to align the output vector with standard computer vision conventions (X -right, Y -down, Z -forward):

$$\mathbf{t} = \frac{3\text{D} \left(\mathbf{A}_{CV} \begin{bmatrix} \hat{\mathbf{v}}_{\text{cam}} \\ 0 \end{bmatrix} \right)}{\left\| 3\text{D} \left(\mathbf{A}_{CV} \begin{bmatrix} \hat{\mathbf{v}}_{\text{cam}} \\ 0 \end{bmatrix} \right) \right\|_2} = (t_x, t_y, t_z)^\top \in \mathbb{S}^2 \quad (3)$$

where $\mathbf{t}$ represents the final normalized 3D thrust unit vector associated with the target airframe.

2.1.2 Pipeline Synchronization and Domain Randomization

Scene graph transformations in USD stage evaluations can exhibit a single-frame pipeline latency relative to Replicator’s RGB sensor annotators. To eliminate spatial misalignment, our custom writer buffers the USD transform matrix $\mathbf{M}_{\text{world}}^{(N)}$ at frame N and evaluates it against image data $I^{(N)}$, ensuring exact frame synchronization.

To encourage feature robustness, we apply domain randomization during rendering: target drone position $P \sim \mathcal{U}((2, 2, 8), (6, 8, 16))$ meters, body orientation (pitch, roll,

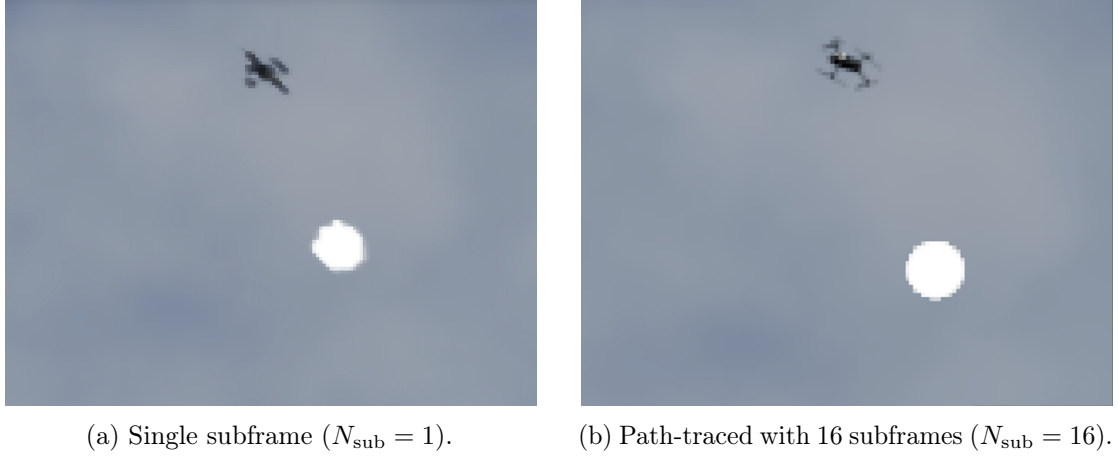


Figure 3: **Impact of sub-pixel sample accumulation on synthetic render quality.** (a) Single-sample rendering exhibits spatial aliasing along high-contrast boundaries and thin airframe struts. (b) Accumulating 16 path-traced subframes per pixel smooths edge transitions through sub-pixel anti-aliasing, preserving structural details.

yaw), and solar lighting azimuth/elevation angles are uniformly randomized across frames. Scene rendering utilizes path tracing with 16 subframes per pixel to generate photorealistic reflections and self-shadowing. As shown in Figure 3, accumulating 16 sub-pixel samples drastically mitigates high-frequency spatial aliasing, eliminating staircasing artifacts along high-contrast boundaries and preserving fine geometries such as propellers and airframe struts.

2.2 ArUco Markers and Perspective Projection

Our second approach establishes a traditional baseline by utilizing physical ArUco fiducial markers rigidly attached to the quadrotor chassis. To extract the orientation, we detect the marker corners and estimate the relative camera pose using OpenCV’s Perspective- n -Point (PnP) solver.

A well-documented limitation of utilizing coplanar feature points for PnP estimation is the planar pose ambiguity problem. Depending on the viewing angle, sensor noise, and distance, the optimization solver can frequently oscillate between two mathematically valid but physically distinct local minima. This manifests as a spurious reflection of the normal vector along the optical axis, causing the estimated Z -axis of the marker, and consequently the derived thrust vector, to erroneously jump by approximately 180° for isolated frames.

Because quadrotors are governed by continuous flight dynamics and rotational inertia, it is physically impossible for the airframe to instantaneously invert its spatial orientation within the minimal Δt of a single video frame. To correct these tracking artifacts, we implement a temporal consistency filter across a sliding observation window. We calculate the angular divergence between the estimated thrust vectors of adjacent frames. If a transient $\sim 180^\circ$ rotation is detected at time t , but the adjacent frames at $t - 1$ and $t + 1$ exhibit continuous alignment (e.g., both correctly pointing upward in the world frame), we deduce that the solver has collapsed into the ambiguous flipped solution. Consequently, we programmatically invert the Z -axis of the anomalous pose matrix at time t to match its neighbors. This corrective step enforces temporal smoothness on the rotation manifold $SO(3)$, ensuring continuous and physically viable thrust vector trajectories prior to back-projecting the annotations into the camera frame.

These Problems could be lessened by using more ArUco codes around the target

drone. However, this approach would create additional cues not present in real world scenarios. Thus, leading to shortcut learning, a phenomenon where models employ simple, non-robust decision rules in practical tasks, which hinders their generalization and robustness because they learn to rely on the markers instead of the geometry of the airframe. Song et al. [2024]

2.3 MIVE: Markerless Annotation via Visual-Inertial Odometry

Because physical fiducial markers risk biasing visual representations toward artificial high-contrast geometric patterns, we propose the MIVE framework, leveraging mobile Visual-Inertial Odometry (VIO). Using a handheld mobile device running Apple ARKit via the *Stray Scanner* interface, we capture synchronized RGB video along with high-frequency 6DoF VIO camera trajectories and calibrated intrinsic camera matrices.

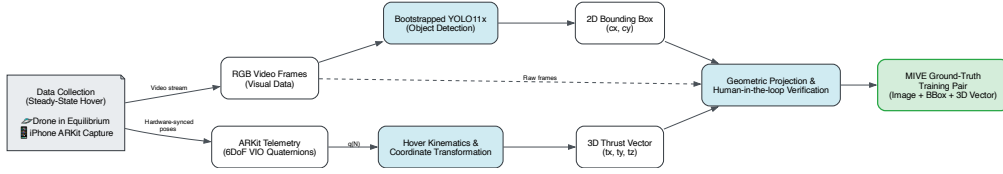


Figure 4: **MIVE Data Generation Pipeline.** Physical RGB frames and ARKit IMU telemetry are hardware-synchronized. A bootstrapped YOLO11x model extracts 2D bounding boxes, while continuous VIO orientation transforms the steady-state hover gravity vector into a continuous 3D thrust vector, yielding a markerless ground-truth training pair.

2.3.1 Aerodynamic Equilibrium and Error Bounds

Under steady-state hover in low-turbulence conditions, the quadrotor resides in aerodynamic force equilibrium where total motor thrust $\mathbf{T}_{\text{body}}$ directly opposes the gravitational acceleration vector $\mathbf{g} = [0, -g, 0]^\top$. In ARKit’s gravity-aligned global frame (where $+Y$ aligns antiparallel to gravity), the ideal thrust vector is static:

$$\mathbf{v}_{\text{world}} = [0.0, \ 1.0, \ 0.0]^\top \quad (4)$$

While active flight controllers introduce micro-corrections to maintain spatial stationarity, flight-log telemetry under calm conditions confirms tilt oscillations remain below $\pm 1.5^\circ$. This establishes an exceptionally tight error bound for passive optical supervision without requiring active motion-capture markers.

2.3.2 Orientation Transformation via VIO Manifolds

For each video frame N , ARKit provides a hardware-synchronized unit quaternion $\mathbf{q}^{(N)} = [q_x, q_y, q_z, q_w]^\top \in \mathbb{S}^3$ referenced to the gravity-aligned origin. Because gravity is strictly observable via accelerometer bias estimation in the VIO backend, the vertical reference axis does not suffer from the unobservable drift associated with visual yaw tracking. We map $\mathbf{q}^{(N)}$ to an orthonormal rotation matrix $\mathbf{R}_{\text{cam} \rightarrow \text{world}} \in \text{SO}(3)$ and obtain the world-to-camera rotation via transposition:

$$\mathbf{R}_{\text{world} \rightarrow \text{cam}} = \mathbf{R}_{\text{cam} \rightarrow \text{world}}^\top \quad (5)$$

The continuous 3D thrust unit vector in the instantaneous camera coordinate frame is then derived directly via rotation and subsequent normalization:

$$\mathbf{t}_{\text{cam}} = \mathbf{R}_{\text{world} \rightarrow \text{cam}} \mathbf{v}_{\text{world}}, \quad \mathbf{t} = \frac{\mathbf{t}_{\text{cam}}}{\|\mathbf{t}_{\text{cam}}\|_2} = (t_x, t_y, t_z)^\top \in \mathbb{S}^2 \quad (6)$$

2.3.3 Automated Bounding Box Extraction via Bootstrapped YOLO11x

To extract the 2D spatial coordinates bounding the quadrotor across thousands of captured frames, manual annotation is intractable. Instead, we employ a bootstrapping approach. A small, diverse seed dataset of initial frames is manually annotated with tight 2D bounding boxes. We utilize this seed data to fine-tune a high-capacity YOLO11x object detector, chosen for its superior feature extraction capabilities and robust generalization.

Once fine-tuned, the YOLO11x model is deployed to run offline inference across the remaining unannotated video sequences. This automated pipeline generates precise bounding box centers $(c_x^{\text{box}}, c_y^{\text{box}})$ for the target airframe across all frames, significantly scaling the dataset volume while maintaining high localization accuracy.

2.3.4 Geometric Projection and Human-in-the-Loop Verification

Data collection is executed by moving the sensor along continuous orbital trajectories around the stationary airframe, sweeping elevation angles from -90° to $+90^\circ$ and distances from 1.0 m to 8.5 m to populate a diverse sample distribution across $\mathbb{S}^2$.

To verify annotation fidelity, we project the derived 3D thrust vector onto the 2D image plane using the calibrated camera intrinsics matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (7)$$

We back-project the bootstrapped YOLO11x bounding box center $(c_x^{\text{box}}, c_y^{\text{box}})$ to 3D camera coordinates $\mathbf{P}_{3D}$ using the scale factor Z , derived from the mobile device’s onboard LiDAR depth measurements:

$$\mathbf{P}_{3D} = \left[\frac{(c_x^{\text{box}} - c_x) \cdot Z}{f_x}, \frac{(c_y^{\text{box}} - c_y) \cdot Z}{f_y}, Z \right]^\top \quad (8)$$

The 3D endpoint of a directional vector of length L is $\mathbf{P}_{\text{tip}} = \mathbf{P}_{3D} + L \cdot \mathbf{t}$. Using pinhole projection, its 2D coordinates $(u_{\text{tip}}, v_{\text{tip}})$ are:

$$u_{\text{tip}} = \frac{[\mathbf{P}_{\text{tip}}]_x \cdot f_x}{[\mathbf{P}_{\text{tip}}]_z} + c_x, \quad v_{\text{tip}} = \frac{[\mathbf{P}_{\text{tip}}]_y \cdot f_y}{[\mathbf{P}_{\text{tip}}]_z} + c_y \quad (9)$$

An interactive review interface overlays the projected vector onto candidate frames. This allows human annotators to quickly inspect the automated YOLO11x bounding boxes alongside the VIO-derived thrust vectors, rejecting any frames exhibiting transient aerodynamic drift or localization failures.

3 Neural Network Architecture

Rather than proposing a state-of-the-art pose estimator, our goal is to design a lightweight, real-time baseline architecture to serve as a uniform perception benchmark across our different dataset generation strategies. To enable this, we introduce an end-to-end architecture built on YOLO11n. Our custom *DroneThrustHead* extends the network to simultaneously predict 2D bounding boxes, class probabilities, and continuous 3D thrust vectors.

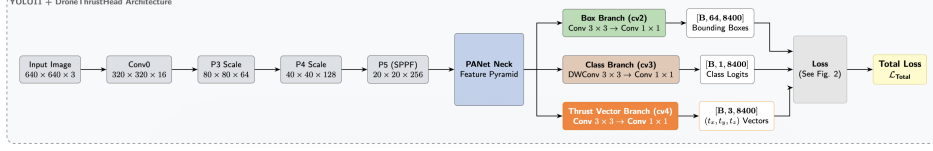


Figure 5: **YOLO11 + DroneThrustHead Architecture.** Multi-scale feature maps from the PANet neck are processed by three parallel decoupled branches predicting 2D bounding boxes, class logits, and 3D thrust vectors.

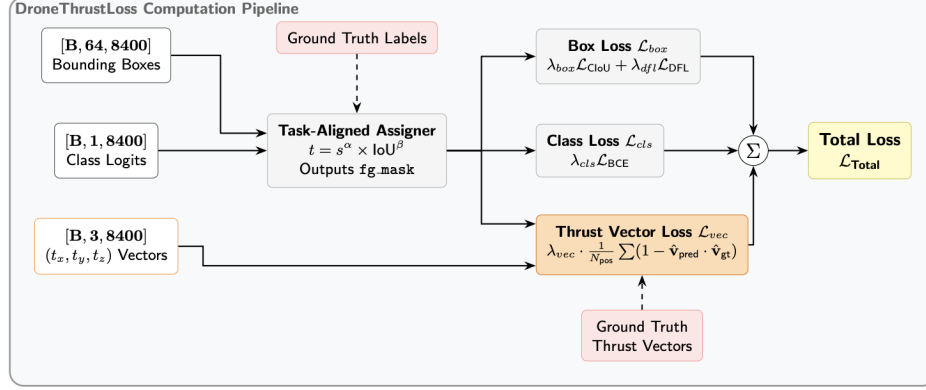


Figure 6: **DroneThrustHead Decoupled Output Structure.** The head isolates spatial bounding box regression, classification, and continuous 3D thrust vector prediction for multi-task optimization.

3.1 Backbone and Neck Architecture

Given an input RGB image $I \in \mathbb{R}^{640 \times 640 \times 3}$, the backbone extracts multi-scale feature hierarchies using convolutional layers, C3k2 residual blocks, and a Spatial Pyramid Pooling Fast (SPPF) module. Feature representations are extracted at three scales: P3 ($80 \times 80 \times 64$), P4 ($40 \times 40 \times 128$), and P5 ($20 \times 20 \times 256$). The Path-Aggregation Network (PANet) neck fuses these features via top-down and bottom-up pathways to preserve spatial detail.

3.2 DroneThrustHead: Multi-Task Decoupled Head

At each scale $i \in \{3, 4, 5\}$, neck features are routed into our decoupled *DroneThrustHead*. Across a combined grid of 8,400 anchor locations, the network outputs three decoupled prediction tensors:

- **Bounding Box Branch:** Outputs 64 Distribution Focal Loss (DFL) channels per anchor, which are decoded into 4 spatial bounding box boundary offsets $[B, 4, 8400]$.
- **Classification Branch:** Predicts the target objectness and class logit tensor of shape $[B, 1, 8400]$.
- **Thrust Vector Branch:** A specialized regression head composed of a 3×3 convolution followed by a 1×1 pointwise convolution. It directly regresses unnormalized 3D directional components (t_x, t_y, t_z) per anchor, yielding a $[B, 3, 8400]$ tensor.

3.3 Task-Aligned Assignment and DroneThrustLoss

We employ a Task-Aligned Assigner to dynamically match candidate anchors against ground-truth targets using an alignment metric $t = s^\alpha \times \text{IoU}^\beta$ (where s denotes classifi-

cation score), generating a binary foreground mask of positive anchor indices.

The total multi-task loss is defined as:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{vec}}\mathcal{L}_{\text{vec}} \quad (10)$$

with empirical weights set to $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$, and $\lambda_{\text{vec}} = 2.0$. The bounding box loss $\mathcal{L}_{\text{box}}$ aggregates CIoU and DFL losses, while $\mathcal{L}_{\text{cls}}$ is supervised via Binary Cross-Entropy.

For the thrust vector branch, raw vector predictions $\mathbf{v}_{\text{pred}} \in \mathbb{R}^3$ at positive foreground anchors are L_2 -normalized onto the unit sphere $\mathbb{S}^2$:

$$\hat{\mathbf{v}}_{\text{pred}} = \frac{\mathbf{v}_{\text{pred}}}{\|\mathbf{v}_{\text{pred}}\|_2 + \epsilon}, \quad \hat{\mathbf{v}}_{\text{gt}} = \frac{\mathbf{v}_{\text{gt}}}{\|\mathbf{v}_{\text{gt}}\|_2} \quad (11)$$

where $\epsilon = 10^{-7}$ ensures numerical stability. The orientation loss $\mathcal{L}_{\text{vec}}$ is evaluated via mean cosine distance across all N_{pos} assigned positive anchors:

$$\mathcal{L}_{\text{vec}} = \frac{1}{N_{\text{pos}}} \sum_{i=1}^{N_{\text{pos}}} (1 - \hat{\mathbf{v}}_{\text{pred},i} \cdot \hat{\mathbf{v}}_{\text{gt},i}) \quad (12)$$

3.4 Implementation and Training Details

To ensure reproducibility across all benchmarked dataset configurations, we standardized our training hyperparameters. The YOLO11n backbone was initialized with custom weights pretrained on a drone dataset, while the custom *DroneThrustHead* was initialized randomly.

All model variants were trained for 100 epochs with a batch size of 16 on a single NVIDIA RTX 5090 GPU. We optimized the network utilizing the AdamW optimizer with an initial learning rate of 1×10^{-3} and a weight decay of 5×10^{-4} .

During training, input images were resized to 640×640 pixels while preserving aspect ratios via letterbox padding. Crucially, standard spatial augmentations (such as random horizontal flips and mosaic) were explicitly disabled to prevent the corruption of the ground-truth 3D thrust vector geometry, restricting data augmentation exclusively to photometric transformations (HSV color jitter).

4 Evaluation

To assess the performance of each data generation paradigm, we use our modified YOLO11n architecture equipped with *DroneThrustHead* as a uniform perception benchmark. We train identical model configurations across four dataset settings and evaluate each model on an independent synthetic Isaac Sim test split ($N = 400$) and a real-world Camera-IMU test set ($N = 1289$).

4.1 Evaluation Metrics

Orientation accuracy is quantified by computing the Mean Angular Error (MAE) between the predicted 3D unit thrust vector $\hat{\mathbf{v}}$ and the ground-truth unit vector $\mathbf{v}$:

$$\theta_{\text{err}} = \arccos \left(\frac{\hat{\mathbf{v}} \cdot \mathbf{v}}{\|\hat{\mathbf{v}}\| \|\mathbf{v}\|} \right) \times \frac{180^\circ}{\pi} \quad (13)$$

In addition, we report the True Positive (TP) detection rate on the target airframe at an IoU threshold of 0.5.

We benchmark four distinct training configurations:

Table 1: **Cross-Dataset Evaluation Matrix.** Performance comparison of model variants evaluated on synthetic Isaac Sim validation data ($N = 400$) and real-world Camera-IMU test sequences ($N = 1289$).

Model Variant	Evaluation Dataset	Evaluated (N)	TP (%)	MAE (θ_{err})
Synthetic Data Only	Isaac Sim	400	96.5%	7.79°
	Real Data	1289	64.9%	50.89°
ArUco Data Only	Isaac Sim	400	8.7%	118.73°
	Real Data	1289	36.8%	57.76°
MIVE Data Only	Isaac Sim	400	56.2%	91.16°
	Real Data	1289	84.1%	18.15°
All Images	Isaac Sim	400	94.0%	21.69°
	Real Data	1289	88.2%	36.86°

1. **Synthetic Data Only:** Model trained exclusively on photorealistic Isaac Sim synthetic renders.
2. **ArUco Data Only:** Model trained exclusively on physical drone images containing ArUco fiducials.
3. **MIVE Data Only:** Model trained exclusively on Camera-IMU VIO telemetry data generated via MIVE.
4. **All Images:** Model trained on a unified dataset combining Synthetic, ArUco, and MIVE data.

4.2 Empirical Benchmark Matrix

Table 1 summarizes the performance across all evaluated dataset configurations and test splits.

4.3 Diagnostic Discussion and Insights

The empirical results reveal several key insights regarding training data distributions and representation learning:

1. **Visual Shortcut Learning in Fiducial Datasets:** The model trained exclusively on ArUco-tagged drones generalized poorly to unmarked target airframes, achieving a detection recall of only 36.8% on real test imagery and 8.7% in simulation. The feature extractor over-indexed on the planar, high-contrast black-and-white fiducial patterns rather than learning intrinsic geometric features of the drone airframe.
2. **Efficacy of MIVE Supervision:** The model trained using MIVE Camera-IMU telemetry demonstrated the strongest real-world performance, achieving an 18.15° **Mean Angular Error** alongside an 84.1% **True Positive rate**. Learning from physical multi-view imagery under natural lighting allows the network to exploit subtle shading and airframe asymmetries for orientation regression.
3. **Sim-to-Real Domain Shift:** Although the synthetic-only model performed with high accuracy on simulated test data (MAE = 7.79°, TP = 96.5%), its angular error degraded to 50.89° on real-world test scenes. This error increase highlights the persistent domain gap between path-traced CAD materials and real-world optical

characteristics. Additionally the lack of complex objects, like tree branches, may have negatively influenced the generalization of this approach.

4. **Effects of Dataset Aggregation:** Training on a composite dataset (**All Images**) yielded the highest real-world detection rate (88.2%), but produced a higher angular error ($\text{MAE} = 36.86^\circ$) than pure MIVE supervision (18.15°). Incorporating fiducial marker imagery introduced conflicting visual cues, degrading fine-grained thrust vector regression.

5 Conclusion and Future Work

In this work, we investigated the critical bottleneck of ground-truth data generation for monocular 3D thrust vector regression. We introduced MIVE, a novel, markerless data generation paradigm that leverages mobile Visual-Inertial Odometry (VIO) telemetry and hover force equilibrium to capture continuous orientation data without expensive motion-capture infrastructure or visually intrusive markers.

To evaluate our pipeline, we integrated a multi-task *DroneThrustHead* module into YOLO11n, establishing a uniform perception benchmark to compare our MIVE pipeline against traditional synthetic and fiducial-based datasets. Our comparative evaluation yields several vital insights for real-world quadrotor tracking:

- Physical fiducial markers (ArUco boards) severely compromise representation learning. The network over-indexes on the marker patterns, leading to catastrophic failure on unmarked operational airframes ($\text{TP}=36.8\%$).
- Synthetic generation via NVIDIA Isaac Sim provides excellent baseline localization ($\text{TP}=96.5\%$) but is heavily constrained by the sim-to-real domain gap, resulting in substantial orientation degradation ($\text{MAE} = 50.89^\circ$) on real-world test scenes.
- Our MIVE Camera-IMU telemetry fusion provides the most robust and effective real-world supervision. By forcing the network to learn intrinsic geometric and shading cues of the airframe, this dataset strategy achieved an 18.15° **Mean Angular Error** and an 84.1% **True Positive rate** in real-world testing.

Ultimately, our findings demonstrate that resolving the sim-to-real and shortcut-learning bottlenecks through intelligent, scalable real-world data collection is paramount for the deployment of robust vision-based aerial interceptors.

In future work, we aim to systematically quantify the effects of camera intrinsic parameters such as focal length distortions and the absence of depth information on the absolute accuracy of the generated orientation data. Furthermore, having established the efficacy of our MIVE data generation pipeline, future research will explore its integration with specialized, state-of-the-art neural architectures designed explicitly for continuous monocular pose estimation, moving beyond the generalized object detection baseline utilized in this study.

Acknowledgements

The author would like to thank Frederik Veit Kepler (MIT) for valuable discussions and feedback on early drafts of this work.

References

- Yannick Bukschat and Marcus Vetter. Efficientpose: An efficient, accurate and scalable end-to-end 6d multi object pose estimation approach. *arXiv preprint arXiv:2011.04307*, 2020.
- Hyun-Ki Jung et al. Real-time target tracking and interception for autonomous uavs using single-stage detectors. *IEEE Robotics and Automation Letters*, 2025.
- Yann Labbé et al. Megapose: 6d pose estimation of novel objects via render & compare. *CoRL*, 2022.
- M. Lachance et al. The case for better ground truth in aerial robotics: Datasets and benchmarks. *Robotics and Autonomous Systems*, 2023.
- X. Liu et al. A comprehensive survey on monocular 6dof object pose estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- T. Nguyen et al. Gigapose: Fast and robust novel object pose estimation via one-shot template matching. *CVPR*, 2024.
- Mahdi Rad and Vincent Lepetit. Bb8: A scalable, accurate, robust to partial occlusion method for predicting the 3d poses of challenging objects without using depth. In *ICCV*, 2017.
- Carmelo Scribano, Iacopo Ferrari, Giorgia Franchini, Elena Govi, Davide Sapienza, Tobia Poppi, Micaela Verucchi, and Marko Bertogna. Generative data augmentation for aruco-free rgb-based 6-dof object pose estimation. *Journal of Imaging*, 12(6), 2026. ISSN 2313-433X. doi: 10.3390/jimaging12060244. URL <https://www.mdpi.com/2313-433X/12/6/244>.
- Ivan Shugurov et al. Osop: A multi-stage pipeline for object pose estimation. *ECCV*, 2022.
- Rui Song, Yingji Li, Lida Shi, Fausto Giunchiglia, and Hao Xu. Shortcut learning in in-context learning: A survey, 2024. URL <https://arxiv.org/abs/2411.02018>.
- Jiaming Sun, Zihao Wang, Siyu Zhang, Xingyi He, Hongcheng Zhao, Guofeng Zhang, and Xiaowei Zhou. Onepose: One-shot object pose estimation without cad models. *CVPR*, 2022.
- Gu Wang et al. Gdr-net: Geometry-guided direct regression network for monocular 6d object pose estimation. In *CVPR*, 2021.